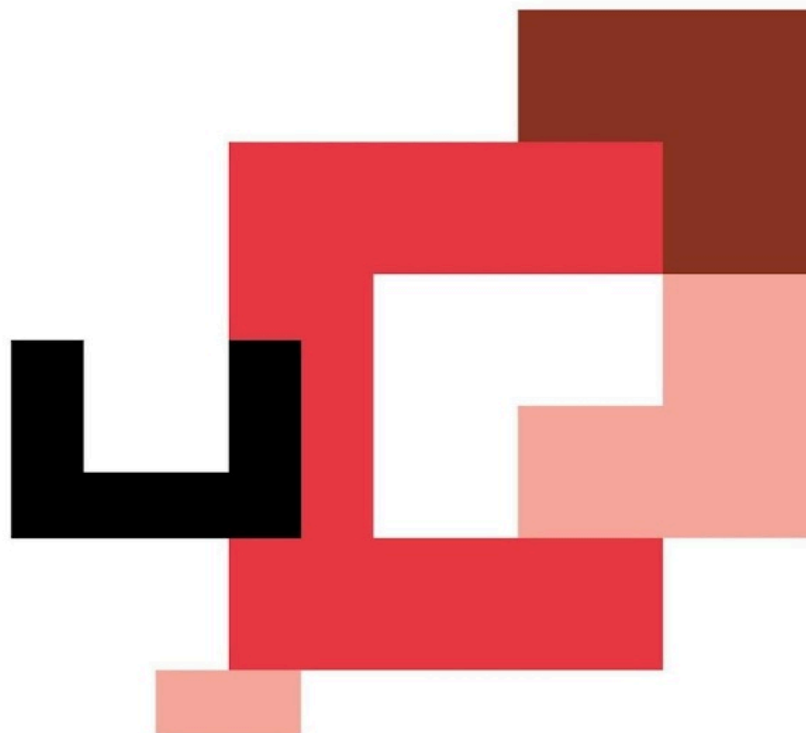




FULL STACK DEVELOPER + IA

Cheatsheet Destructuring, Spread, Bucles Avanzados y LocalStorage



Destructuring (objetos)

1) Extraer propiedades

```
const user = { id: 1, name: "Ana", role: "admin" };
```

```
const { id, name } = user;
```

```
console.log(id, name);
```

2) Renombrar variables

```
const { name: username } = user; // username = "Ana"
```

3) Valores por defecto

```
const { city = "Madrid" } = user;
```

4) Destructuring anidado

```
const data = { profile: { email: "a@a.com" } };
```

```
const {  
  profile: { email },  
} = data;
```

```
console.log(email);
```

5) Destructuring en parámetros (muy usado)

```
const printUser = ({ name, role }) => `${name} - ${role}`;
```

```
printUser(user);
```

Destructuring (arrays)

```
const nums = [10, 20, 30];  
  
const [a, b] = nums;           // a=10, b=20  
  
const [, , c] = nums;         // c=30 (saltas posiciones)  
  
const [x = 1, y = 2] = [];    // defaults: x=1, y=2
```

Swap (intercambiar valores):

```
let p = "A";  
  
let q = "B";  
  
[p, q] = [q, p]; // p="B", q="A"
```

Spread ... (arrays)

1) Copiar array (sin mutar)

```
const a = [1, 2, 3];  
  
const b = [...a];
```

2) Añadir elementos (sin mutar)

```
const next = [...a, 4];    // al final  
  
const next2 = [0, ...a];   // al inicio
```

3) Concatenar arrays

```
const c = [...[1, 2], ...[3, 4]]; // [1,2,3,4]
```

Spread ... (objetos)

1) Copiar objeto

```
const user = { name: "Ana", role: "user" };  
  
const copy = { ...user };
```

2) Actualizar propiedad (sin mutar)

```
const updated = { ...user, role: "admin" };
```

3) Merge (unir objetos)

```
const a = { x: 1, y: 2 };  
  
const b = { y: 99, z: 3 };  
  
const merged = { ...a, ...b }; // y se sobrescribe por el último: 99
```

Rest ... (capturar “el resto”)

En objetos:

```
const user = { id: 1, name: "Ana", role: "admin" };  
  
const { role, ...publicData } = user; // publicData: {id, name}
```

En arrays:

```
const nums = [1, 2, 3, 4];  
  
const [first, ...rest] = nums; // first=1, rest=[2,3,4]
```

En parámetros:

```
const sumAll = (...nums) => nums.reduce((acc, n) => acc + n, 0);  
  
sumAll(1, 2, 3); // 6
```

Bucles avanzados: for...of

Para arrays (simple y recomendado):

```
const items = ["a", "b", "c"];

for (const item of items) {

  console.log(item);

}
```

Con índice (usando `entries()`):

```
for (const [i, item] of items.entries()) {

  console.log(i, item);

}
```

Bucles avanzados: `for...in` (objetos)

Úsalo para recorrer claves de objetos:

```
const user = { name: "Ana", role: "admin" };

for (const key in user) {

  console.log(key, user[key]);

}
```

Alternativa moderna (muy usada):

```
for (const [key, value] of Object.entries(user)) {

  console.log(key, value);

}
```

`forEach` vs `map` vs `filter`

```
const nums = [1, 2, 3];
```

// forEach: recorre, NO devuelve array

```
nums.forEach((n) => console.log(n));
```

// map: transforma, DEVUELVE array nuevo

```
const doubled = nums.map((n) => n * 2);
```

// filter: filtra, DEVUELVE array nuevo

```
const evens = nums.filter((n) => n % 2 === 0);
```

LocalStorage: guardar y leer

Guardar:

```
localStorage.setItem("theme", "dark");
```

Leer:

```
const theme = localStorage.getItem("theme"); // "dark" o null
```

Borrar una clave:

```
localStorage.removeItem("theme");
```

Borrar todo:

```
localStorage.clear();
```

LocalStorage guarda strings. Para objetos/arrays: **JSON.stringify** y **JSON.parse**.

LocalStorage con objetos/arrays (patrón real)

Guardar array:

```
const tasks = [
```

```
{ id: 1, text: "Estudiar", done: false },  
  { id: 2, text: "Entrenar", done: true },  
];  
  
localStorage.setItem("tasks", JSON.stringify(tasks));
```

Leer array (con fallback seguro):

```
const raw = localStorage.getItem("tasks");  
  
const tasksFromStorage = raw ? JSON.parse(raw) : [];
```

Patrón CRUD simple con LocalStorage

Helpers:

```
const KEY = "tasks";  
  
const loadTasks = () => {  
  const raw = localStorage.getItem(KEY);  
  return raw ? JSON.parse(raw) : [];  
};  
  
const saveTasks = (tasks) => {  
  localStorage.setItem(KEY, JSON.stringify(tasks));  
};
```

Crear:

```
const tasks = loadTasks();  
  
tasks.push({ id: Date.now(), text: "Nueva", done: false });
```

```
saveTasks(tasks);
```

Actualizar (toggle):

```
const tasks = loadTasks();
```

```
const updated = tasks.map((t) =>
```

```
  t.id === 1 ? { ...t, done: !t.done } : t
```

```
);
```

```
saveTasks(updated);
```

Borrar:

```
const tasks = loadTasks();
```

```
const next = tasks.filter((t) => t.id !== 1);
```

```
saveTasks(next);
```

Resumen rápido

- **Destructuring:**
 - **Objetos:** `const { a, b } = obj`
 - **Arrays:** `const [a, b] = arr`
- **Spread:**
 - **Arrays:** `[...arr, x]`
 - **Objetos:** `{ ...obj, a: 2 }`
- **Rest:**
 - **Objetos:** `const { a, ...rest } = obj`
 - **Params:** `(...args) => {}`
- **Bucles:**
 - **Arrays:** `for...of, arr.entries()`
 - **Objetos:** `for...in, Object.entries()`
- **LocalStorage:**
 - `setItem/getItem/removeItem/clear`
 - **Objetos/arrays:** `JSON.stringify + JSON.parse`